

Transformer-Based Model for Cold Start Mitigation in FaaS Architecture

Alexandre Savi Fayam Mbala Mouen, Jerry Lacmou Zeutouo, Vianney Kengne Tchendji *Member, IEEE*

Abstract—Serverless architectures, particularly the Function as a Service (FaaS) model, have become a cornerstone of modern cloud computing due to their ability to simplify resource management and enhance application deployment agility. However, a significant challenge remains: the cold start problem. This phenomenon occurs when an idle FaaS function is invoked, requiring a full initialization process, which increases latency and degrades user experience. Existing solutions for cold start mitigation are limited in terms of invocation pattern generalization and implementation complexity. In this study, we propose an innovative approach leveraging Transformer models to mitigate the impact of cold starts in FaaS architectures. Our solution excels in accurately modeling function initialization delays and optimizing serverless system performance. Experimental evaluation using a public dataset provided by Azure demonstrates a significant reduction in cold start times, reaching up to 79% compared to conventional methods.

Index Terms—serverless, FaaS, cold start, deep learning, time-series forecasting, Transformers, latency, performance optimization

I. INTRODUCTION

THE rapid emergence of cloud computing has transformed the digital industry by enabling immediate and flexible access to virtualized computing resources. Services such as storage, computing, and networking allow organizations to reduce costs [1] while benefiting from scalable, on-demand infrastructure managed by external providers. At the core of this evolution is the Function as a Service (FaaS) model, an innovative solution that allows functions to be deployed in response to specific events without requiring the management of underlying resources. This model, foundational to serverless architectures, provides increased flexibility and usage-based billing, reducing both costs and operational complexity for developers. Serverless architectures, built upon the FaaS model, abstract infrastructure management, allowing developers to focus solely on the code and features of their applications [2]. This approach optimizes application scalability and simplifies infrastructure management, which is a notable advantage in an era of massive automation. Organizations increasingly seek streamlined processes to enhance responsiveness and efficiency, making serverless architectures a key driver of modern digital transformation.

However, despite its numerous advantages, FaaS poses specific challenges [3], particularly the cold start issue. Cold start

latency arises when functions must be loaded into containers from an idle state [4]. This occurs when a function has not been invoked for some time, necessitating a complete reinitialization of the runtime environment—a step that can introduce significant delays. While tolerable in certain applications, cold starts become problematic in high-performance or latency-sensitive environments. Cold start impacts user experience and the responsiveness of serverless applications, especially in scenarios with highly variable workloads. This limitation represents a major barrier to the widespread adoption of FaaS for latency-sensitive applications, necessitating robust solutions to mitigate its effects and improve serverless platform performance.

In recent years, various studies have focused on optimizing resource management and reducing cold starts in FaaS environments. However, most existing work relies on static allocation techniques or heuristic workload analyses, which fail to account for the highly dynamic nature of FaaS platforms. Moreover, few solutions leverage advancements in artificial intelligence (AI) and deep learning, which offer powerful tools to predict workloads and proactively manage resource needs. To address these limitations, our research proposes a deep learning-based approach, specifically using Transformers, to effectively mitigate cold start delays and frequency in FaaS architectures. The choice of Transformers is justified by their ability to capture complex relationships within temporal data, enabling precise function usage predictions and proactive resource management. By integrating a Transformer-based model, our approach aims to reduce function startup delays while optimizing resource consumption.

This paper makes several significant contributions, including :

- 1) Implementation of a Transformer-based model for cold start mitigation, incorporating usage predictions that facilitate proactive resource allocation. Results demonstrate substantial improvements in latency reduction and prediction accuracy, validated by standard evaluation metrics ;
- 2) Performance evaluation through simulations on the OpenWhisk FaaS platform and tests on real-world datasets. Our experiments show improvements in prediction accuracy, cold start delay reduction, and resource optimization, confirming the validity and effectiveness of our approach.

The remainder of this paper is organized as follows :

- Section II – FaaS & Cold Start : Provides an in-depth exploration of the Function as a Service concept, its

Manuscript received XX, 20XX; revised XX, 20XX.

Alexandre S.F.M.M. and Vianney K.T. are with Departement of Mathematics and Computer Science, University of Dschang, Dschang, Cameroon

Jerry L.Z. is with the MIS Labotory, University of Picardie Jules Verne, 33 rue Saint-Leu, 80039 Amiens Cedex, France.

advantages for application deployment, and the cold start limitations inherent in this architecture.

- Section III – Literature Review : Analyzes existing research on workload prediction and cold start mitigation, discussing common resource management approaches and identifying the limitations of traditional methods.
- Section IV – Proposed Approach : Introduces our predictive approach to cold start mitigation, explaining the use of deep learning models, particularly Transformers, to address this challenge.
- Section V – Evaluation : Describes our evaluation methodology and experimental results, analyzing the performance of our model on real-world and simulated datasets.
- Conclusion : Summarizes the contributions of our research, discusses the impact of our results, and proposes future research directions.

II. FAAS & COLD START

With the rapid evolution of cloud computing, the serverless paradigm has revolutionized the way modern applications are designed and deployed. It enables businesses and developers to delegate the management of the underlying infrastructure to cloud service providers while focusing solely on developing the business logic of their applications. Among the various serverless models, Function as a Service (FaaS) has emerged as one of the most promising due to its ability to execute functions in response to events [5], reduce operational costs, and provide automatic scalability. FaaS is an innovative approach that allows developers to deploy and run computing units as standalone functions in a cloud environment. These functions are typically triggered by external events [5], [6], such as HTTP requests, database changes, or IoT (Internet of Things) events. Unlike traditional cloud computing models such as Infrastructure as a Service (IaaS) or Platform as a Service (PaaS), where developers must provision and manage servers or complete environments, FaaS completely abstracts this management, enabling instant deployment and execution of functions on demand.

The core principle of FaaS is based on three main characteristics :

- **Automatic Elasticity** : FaaS is designed to dynamically adapt to workload changes. When demand for a function increases, the cloud provider automatically provisions additional resources to handle the new requests, ensuring applications scale seamlessly with traffic fluctuations ; Conversely, when demand decreases, resources are automatically released.
- **Usage-Based Billing** : unlike traditional models where billing is based on server or virtual machine allocation time, FaaS billing is directly tied to the actual runtime of functions [7]. This approach significantly reduces costs, as developers pay only for the CPU time and memory used during function execution. For example, in scenarios where functions are invoked sporadically, costs are minimized ;

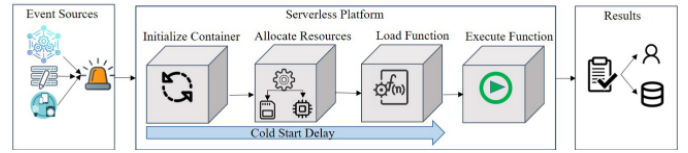


Fig. 1. FaaS execution flow [4]

- **No Infrastructure Management** : the FaaS model simplifies development by eliminating the need for developers to manage servers, operating systems, or network configurations. These tasks are handled entirely by the cloud service provider, enabling developers to focus solely on function logic. This reduces deployment time and streamlines DevOps processes by removing much of the operational complexity associated with infrastructure management.

These features have driven the widespread adoption of FaaS in environments where scalability, flexibility, and cost-effectiveness are priorities, such as in microservices-based applications. In the FaaS model, each function is triggered by a specific event. A function is a stateless computing unit that executes precise logic and returns a result in response to that event. Unlike other cloud paradigms, FaaS does not store data between function executions. This stateless behavior is essential for automatic scaling, as it allows each request to be processed in isolation, independently of others.

Figure 1 illustrates the general process of executing a function in a FaaS architecture [8], [9]:

- 1) **Event Triggering** : an event from an external source, such as an HTTP request or an IoT notification, is captured by the cloud provider ;
- 2) **Container Initialization** : if no active instance of the function exists, the system provisions a new container to execute the function ;
- 3) **Dependency Loading** : the function code and associated libraries are loaded into the container. This process may take time depending on the function's complexity and the volume of required dependencies ;
- 4) **Function Execution** : once the container is ready, the function executes and returns the result to the event source (e.g., an end user via an HTTP API) ;
- 5) **Resource Release** : at the end of the function's execution, the container and all its resources will be released. This is known as scale-to-zero [10], [11], an innovative feature in FaaS that helps reduce costs [12].

This sequence of tasks is transparent to the developer but has a direct impact on performance, particularly when addressing cold starts. Cold start is one of the primary challenges in FaaS architectures. It occurs when a function is invoked for the first time or after a long period of inactivity. In such scenarios, the cloud service provider must allocate a new container to host the function and initialize the runtime environment, resulting in additional latency before the function can respond to requests [13]–[15]. Cold starts can significantly impact the overall performance of applications in FaaS architectures, especially for latency-sensitive applications or those with real-

time high-availability requirements [12], [16]. Some scenarios where cold start can pose a problem include :

- **Real-Time Applications** : in applications such as financial transaction processing systems or emergency alert systems, even a few milliseconds of latency can have significant consequences. In these contexts, cold starts lasting several seconds can compromise user experience and overall system performance ;
- **Demand Spikes** : when an application experiences a sudden traffic surge, cold starts can occur if existing containers cannot handle the new demand. This results in increased latency as new containers must be provisioned to handle the growing demand ;
- **Public APIs** : for widely used public APIs, excessive latency caused by a cold start can lead to longer response times, potentially affecting thousands of simultaneous users .

Cold start is essentially a zero-scaling problem, where a function instance is decommissioned to save resources when the function is not in use. While this ensures efficient resource usage, restarting the function from scratch requires a certain delay before it becomes ready again. The latency of a cold start varies based on factors [17] such as container size, dependency volumes, and the specific configuration of the cloud platform. For example, a study conducted by measured cold start delays for several popular cloud providers, including AWS Lambda, Google Cloud Functions, and Azure Functions. The results showed that cold starts could take anywhere from 100 milliseconds to several seconds, depending on the complexity of the functions and the service provider. FaaS is undoubtedly one of the most transformative innovations in cloud computing, offering unprecedented flexibility and scalability. However, like any emerging technology, it comes with challenges, particularly the issue of cold starts. This challenge can significantly affect application performance, particularly in environments where latency and responsiveness are critical. Proposed solutions, such as pre-allocated instances, code optimization, and the adoption of lighter runtimes, offer ways to mitigate these impacts. However, continued research is needed to improve these mechanisms and ensure optimal performance in large-scale FaaS architectures.

III. LITERATURE REVIEW

FaaS (Function-as-a-Service) architectures, despite their advantages in scalability and simplified resource management, face the challenge of cold starts. This phenomenon, characterized by the delay required to initialize an idle function, can significantly impact the performance of serverless applications, especially when low latency is critical. This section explores approaches developed to mitigate cold starts, focusing on strategies to reduce the delay and frequency of this phenomenon. We review techniques ranging from container pre-warming to machine learning approaches, highlighting their quantitative outcomes, practical constraints, and limitations.

A. Techniques for Reducing Cold Start Delay

Cold start delay reduction techniques aim to decrease the time required for functions to be ready to handle requests. This

delay can vary depending on container configuration, code volume, and specific dependencies. To address this, several approaches have been proposed in the literature:

- **Container Pre-Warming**: Platforms such as OpenWhisk [18] implement pre-warmed container strategies. The primary idea is to maintain a pool of “ready” containers in memory. Upon a function invocation, a pre-warmed container is immediately assigned, significantly reducing startup delays. OpenWhisk, for instance, uses two pre-warmed containers with a Node.js runtime, achieving startup delay reductions of up to 30% in certain cases, especially for simple functions without heavy dependencies. However, this approach has practical drawbacks. It requires maintaining a pool of containers continuously, leading to resource wastage during low or unpredictable demand. Additionally, it is limited to environments configured for specific programming languages, reducing flexibility.
- **Periodic Function Invocation**: To keep functions active and reduce startup delays, platforms like CloudWatch [19] and Lambda Warmer [20] employ periodic function invocation. These tools trigger functions at regular intervals, irrespective of actual demand, to maintain “warm” containers. This technique reduces startup delays by 20% to 40% for intermittently used applications. However, the primary downside is that it does not adapt to workload variations and may lead to significant resource wastage during prolonged inactivity.
- **Selective Code Optimization - FaaSLight** [21]: introduced by Liu et al., proposes loading only the essential parts of application code to reduce startup latency. By initializing only the necessary code, this technique achieves a startup latency reduction of up to 25%. However, this approach is more suitable for applications with simple architectures and minimal dependencies. For more complex applications requiring multiple modules and dependencies, its effectiveness may be limited.
- **Function Fusion** [17]: A novel approach by Lee et al. involves fusing sequential functions into one. This technique avoids a cold start for the second function by combining both into a single container, reducing the overall workflow delay. Experiments have shown an 18% reduction in startup latency for sequential applications. However, for scenarios requiring parallel functions, this approach may result in extended execution times, limiting its applicability.
- **Proactive Prediction with SARIMA** [9]: Advanced methods like time-series prediction models, such as SARIMA, coupled with an autoscaler, predict workloads based on invocation history to initialize containers ahead of peak demand. This approach reduces startup delays by 22% to 30% in high-demand environments. However, it relies heavily on accurate prediction data and may be ineffective in the face of unexpected workload changes.

In summary, these techniques offer notable reductions in function startup delays but have limitations in terms of implementation complexity and operational costs. For instance,

container pre-warming and periodic invocation require additional resources and are challenging to adapt to highly variable demand environments.

B. Techniques for Reducing Cold Start Frequency

Techniques aimed at reducing cold start frequency focus on preserving containers in memory to minimize the number of cold starts during repeated invocations. This strategy is vital for serverless applications requiring rapid and frequent responses.

- **Temporary Retention of Active Containers:** Platforms such as AWS Lambda [19] and Google Cloud Functions [22] keep containers in a paused state for several minutes after function execution. If a new request arrives during this period, the already-in-memory container is reused, eliminating startup delays and reducing cold start frequency by 15% to 20%. However, this technique increases memory usage and requires optimal thresholds for container release. Poor configurations may result in additional costs without significant benefits.
- **Reinforcement Learning with Q-Learning [23], [24]:** Using reinforcement learning algorithms like Q-learning, it is possible to dynamically predict the optimal number of function instances based on demand. By adjusting container retention frequency based on usage patterns, this technique significantly reduces cold starts while minimizing CPU resource usage. Studies have shown a reduction in cold starts by approximately 40%, with up to 55% CPU savings. However, Q-learning requires extensive training data and can be complex to adapt for generalized applications.
- **Hybrid Histogram Policy :** The Hybrid Histogram Policy introduced by Shahrad et al. [3] is an adaptive approach that dynamically adjusts the duration containers remain active. Using histograms to determine function invocation frequencies and optimal inactivity periods, this policy reduces cold start frequency by up to 35%, especially for functions with irregular demand. However, it is challenging to implement and requires real-time invocation data for precise model adjustments.
- **Workload Prediction with LSTM Neural Networks:** Kumari et al. [25] proposed using LSTM neural networks to predict the number of containers to keep active and the optimal retention duration based on workload patterns. Results indicate a reduction in cold start frequency by 18% to 25%, significantly improving performance for serverless applications with variable demand. However, this approach requires a robust infrastructure to process real-time predictions and extensive historical data for training, limiting its accessibility for certain organizations or projects.
- **In another study similar to Kumari's, Vahidinia et al. [4]** use a reinforcement learning method and LSTM models to determine the number of warm containers and the duration for which they are kept warm.

While these approaches effectively reduce cold start frequency, they are often resource-intensive and costly. Temporary con-

tainer retention and machine learning-based methods require continuous monitoring and adjustments to remain effective. Although machine learning and reinforcement learning methods offer promising solutions, resource optimization is necessary to ensure practical adoption in large-scale environments.

IV. PROPOSED APPROACH

In Function-as-a-Service (FaaS) architectures, cold start is a significant performance obstacle, primarily because it increases latency during the first invocation of a function after a period of inactivity. To mitigate this issue, our study proposes an approach based on Transformers tailored for time-series data, following the architecture developed by Kashif [26] and its specific improvements for temporal data. Transformers provide a distinct advantage over traditional recurrent models (such as LSTM and GRU) by managing temporal dependencies without requiring sequential computations, thereby accelerating training and improving prediction accuracy.

By leveraging an attention mechanism particularly effective for managing long-term dependencies, our model adapts to the variability of workloads in FaaS environments, ensuring optimal resource management and reducing cold starts. The primary innovations introduced by the Transformer model adapted for time-series data include its ability to capture complex relationships and generate probabilistic forecasts, offering a more comprehensive and tailored view of future demands. The choice of Transformers, specifically Kashif's model tailored for time-series, is justified for several reasons :

- **Handling Long-Term Dependencies :** unlike LSTMs, Transformers can efficiently capture long-range relationships through their attention mechanism. This is critical in time-series data, where long-term dependencies significantly influence workload prediction ;
- **Parallelized Computation :** transformers exploit parallelized operations, making training faster and more suitable for large datasets—especially advantageous for FaaS platforms, which analyze vast invocation histories ;
- **Probabilistic Forecasting :** by incorporating probabilistic predictions, our model estimates the uncertainty associated with each forecast. This is essential in cloud environments where precise and well-calibrated forecasts are necessary to avoid resource overprovisioning.

The Transformer model for time-series data by Kashif is designed to process sequentially structured data while integrating multiple feature types (statistical, temporal, contextual) for each data point. The methodology is broken down as follows :

- 1) **Encoding Time-Series Data :** each data point is enriched with a set of temporal and static features :
 - **pastValues :** Historical values of the time series that provide the historical context of the function.
 - **pastTimeFeatures :** temporal characteristics for each pastValues point, such as day of the month or month of the year, enabling the model to detect seasonal patterns or recurrences.

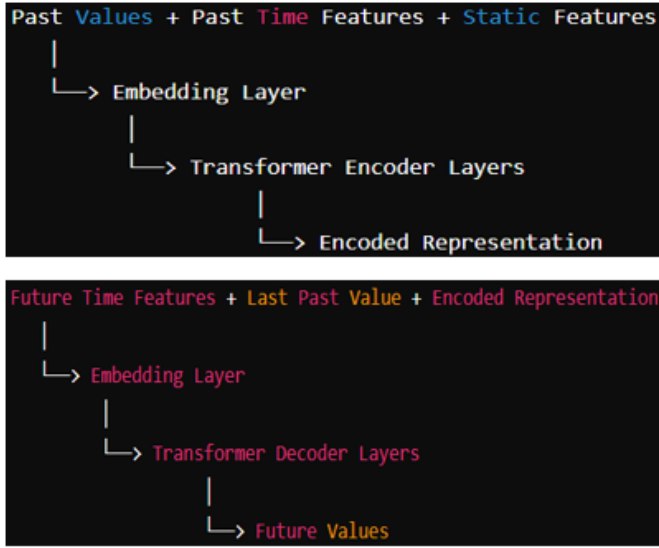


Fig. 2. Input Representation of the Encoder and Decoder

- **staticCategoricalFeatures** : static information associated with the series (e.g., function or client ID), adding fixed context to help the model distinguish between multiple functions or applications.
- 2) **Attention Mechanism** : the core of the Transformer model lies in the attention mechanism. Rather than processing data sequentially, attention assigns weights to points in the sequence based on their relevance to the prediction. This mechanism is vital for detecting nonlinear patterns and capturing interactions across different time periods: leftmargin=1.5cm
 - **Multi-Head Attention** : each attention layer includes several "heads" that can focus on different parts of the time series simultaneously, capturing various aspects of temporal patterns.
 - **Positional Encoding** : since Transformers lack inherent sequential structure like RNNs, positional encoding is added to maintain the sequence order. This encoding helps the model distinguish each point's position within the sequence.
 - 3) **Model Architecture** : the model consists of two main blocks: the encoder and the decoder, specifically modified for time-series forecasting. The architecture draws inspiration from the vanilla Transformer originally designed for natural language processing (NLP). However, instead of processing sequences of words through semantic embeddings, our model adapts the inputs to handle numerical time-series data, as illustrated in the accompanying figure 2. In NLP, inputs are token embeddings combined with positional encodings. Here, pastValues, pastTimeFeatures, and staticCategoricalFeatures replace textual embeddings. These features are projected into a latent space via dedicated embedding layers, enabling the model to capture temporal dynamics (e.g., historical trends, seasonality) and static context (e.g., function identifiers). The encoder fuses these represen-

tations to generate a global historical context, while the decoder leverages futureTimeFeatures and the last observed value to predict future invocations. This adaptation of inputs, combined with multi-head attention, allows the model to process time-series data with the same flexibility as the vanilla Transformer processes sentences, but tailored to numerical and cyclical patterns inherent to FaaS workloads. leftmargin=1.5cm

- **Encoder** : leftmargin=1cm
 - **Input** : the encoder takes pastValues, pastTimeFeatures, and staticCategoricalFeatures.
 - **Processing** : these inputs are combined to produce an encoded representation summarizing the historical context. Multi-head attention captures long-range temporal dependencies, offering a holistic view of the sequence.
 - **Decoder** : leftmargin=1cm
 - **Input** : the decoder uses futureTimeFeatures (temporal characteristics of each future point) and the last value of pastValues.
 - **Processing** : using the encoded representation and futureTimeFeatures, the decoder generates futureValues (predicted values). These predictions incorporate identified temporal patterns and contextual information provided by static and temporal features.
- 4) **Learning Latent Representations** : unlike LSTMs, which struggle to generalize across multiple series, Kashif's Transformer model learns latent representations for a wide range of time-series data. It captures data distributions, enabling probabilistic and non-point predictions (i.e., forecasts represented as distributions).
 - 5) **Probabilistic Forecasting** : the model does not produce a single forecast but captures a distribution of potential outcomes, estimating uncertainty in the predictions [27]. This feature is crucial for adjusting container management based on probabilities of future demand.

V. EVALUATION

This section details the experimental results of our Transformer model for proactive cold start management in FaaS environments. Two essential dimensions were examined: reducing cold start latency and minimizing the frequency of cold starts. The evaluation is accompanied by a direct comparison with LSTM models, highlighting the significant improvements that Transformers offer in FaaS use cases.

A. Experimental Framework and Model Configuration

A central focus of our experimentation was the handling of HTTP-triggered functions. We based our work on a dataset from Microsoft Azure Functions, collected in July 2019 [3], [28]. This dataset originally contained a wide variety of function invocations, but we specifically filtered out only those triggered by HTTP calls. This choice was motivated by two key factors:

- HTTP-triggered functions are the most unpredictable, displaying large fluctuations in invocation rates ;
- They are also among the most popular invocation patterns in serverless platforms.

To address the two critical cold start dimensions—latency (i.e., the “delay” problem) and frequency (i.e., how often cold starts occur)—we leveraged clustering to better characterize the different invocation behaviors. By focusing on popular/frequently used functions for the cold start “delay” problem and irregular/low-usage functions for the cold start “frequency” problem, we ensured that our model could handle a wide spectrum of workload patterns in FaaS.

1) *Data preprocessing and clustering with DBSCAN* (Figure 3): In our initial data processing phase, we started with 14 separate log files, each representing one day’s worth of Azure Functions invocations. We first filtered the data to keep only HTTP-triggered functions (identified via the “Trigger” field). These 14 files were then merged into a single dataset for uniform processing. To capture the variety of invocation patterns, we applied the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm. This step was crucial for segmenting the data into clusters of functions that exhibited similar invocation patterns. Through this process, we identified 33 distinct clusters of HTTP-triggered functions. Each cluster corresponds to a unique pattern of invocations over time, ranging from highly frequent to very sparse usage. From each cluster, we then selected representative functions for subsequent analysis.

This clustering strategy directly supports the cold start objectives:

- High- or Frequent-Demand Functions : often subject to cold start delay concerns, because predicting their peak loads can help us proactively prepare the runtime environment.
- Irregular or Low-Demand Functions : more relevant to cold start frequency issues, since these functions often remain idle for long periods, leading to more frequent container spin-ups.
- Regular or Fixed-Frequency Functions : interestingly, some functions showed stable and unchanging invocation intervals. Their patterns were so predictable that applying either cold start “delay” or “frequency” mitigation strategies consistently yields very high accuracy.

2) *Transformer model setup and hyperparameters*: After segmenting the data into meaningful clusters, we trained a TimeSeriesTransformers model to forecast invocation patterns. The goal was twofold:

- 1) Predict the timing and volume of incoming requests for popular or frequently used functions (mitigating cold start delay by proactively preparing containers).
- 2) Predict the intervals between function calls for less frequent or highly irregular functions (mitigating cold start frequency by intelligently adjusting container idle windows).

Key hyperparameters and configurations for our Transformer model included:

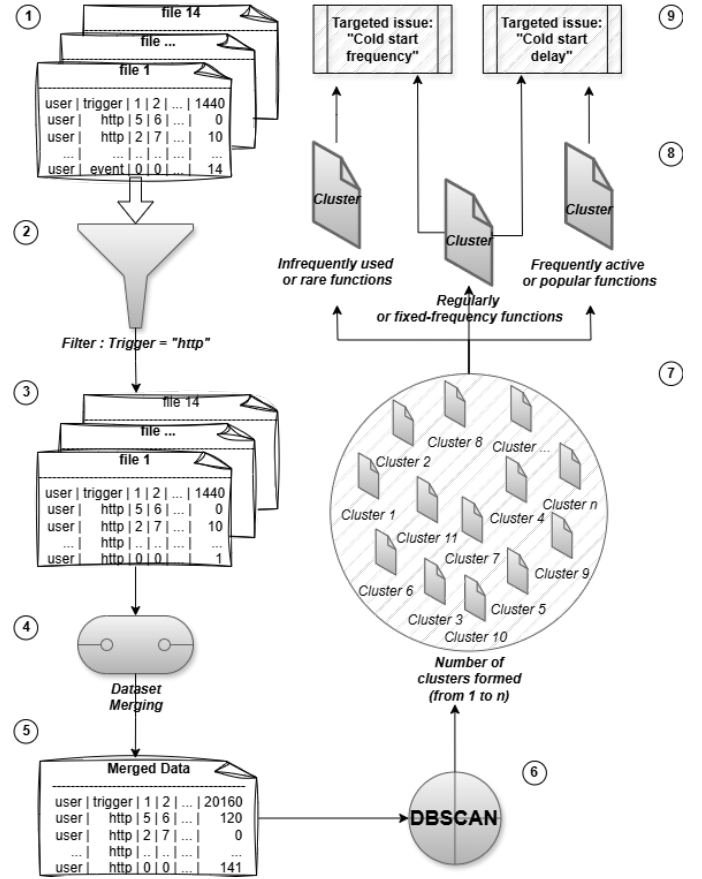


Fig. 3. Global Data Processing Pipeline for Training the Transformer Model

- Contextual Window (contextLength) : 200 points, giving the model a substantial historical view to understand trends and seasonalities.
- Prediction Length (predictionLength) : 100 points, ensuring the model forecasts sufficiently far into the future to anticipate upcoming demand spikes or lulls.
- Number of Encoder/Decoder Layers : 4 layers each, enabling the model to capture both short-term fluctuations and longer-term dependencies.
- Dimension of the Model (dModel) : 32, specifying the size of the hidden representations used in multi-head attention.
- Embedding Dimension (embeddingDimension) : 2, helping to encode categorical features or cluster identifiers.
- Lags Sequence / Cardinality : additional parameters tailored to the time-series nature of the data, enabling the model to incorporate various seasonalities or repeating patterns.

We conducted the model training on Google Colab, leveraging its GPU acceleration for faster computation:

- Python 3 (GPU) runtime, utilizing hardware acceleration for large-scale training.
- System RAM: 12.7 GB, sufficient for loading and processing merged invocation logs.
- GPU RAM: 15.0 GB, enabling the training of deeper Transformer networks without running out of memory.

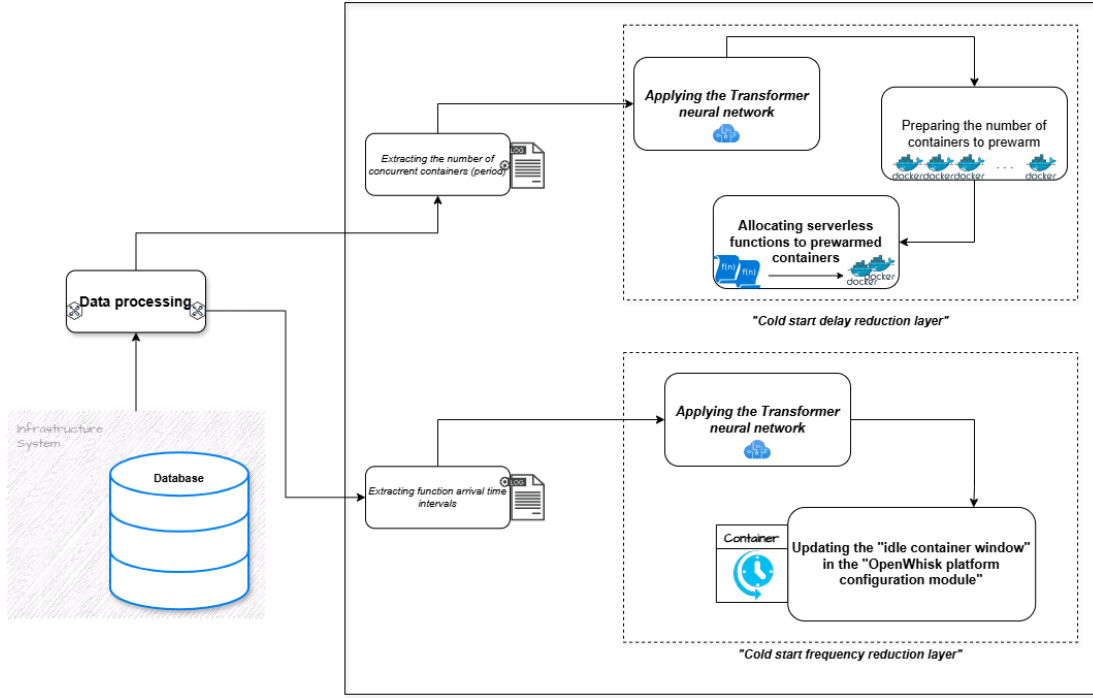


Fig. 4. Proposed Approach: Transformer-Based Model to Mitigate Cold Start

For comparison, we trained a standard LSTM model with a similar number of layers and comparable hidden dimensions. This ensures that the improvement we observe with Transformers is primarily due to the architectural benefits—such as multi-head attention—rather than differences in training data size or compute resources.

3) *Proactive Cold Start management with OpenWhisk (Figure 4)*: To evaluate and deploy our model, we integrated the Transformer-based predictions into an Apache OpenWhisk serverless platform. OpenWhisk natively provides prewarmed containers that are ready to handle incoming requests, mitigating some cold start penalties. However, the default configurations can be somewhat static or purely reactive, which may not suffice for sudden demand spikes or highly irregular usage patterns. Our approach modifies and enhances OpenWhisk’s internal settings:

- **Proactive Allocation of Prewarmed Containers** : by predicting high-demand intervals for frequently invoked functions, we increase the pool of prewarmed containers in advance. This directly addresses the cold start delay problem by ensuring containers are ready when requests arrive.
- **Adaptive Container Inactivity Window** : for functions that are invoked infrequently, we leverage the model’s forecasts of inter-arrival times. This allows us to adjust the “container idle time” or “window of container inactivity” dynamically, reducing the cold start frequency for sporadic invocations.

By combining the Transformers forecasting capabilities with OpenWhisk’s container management, we create an adaptive FaaS platform that addresses both cold start delay and frequency in a unified framework. This configuration ensures

minimal resource waste—only provisioning additional containers when truly needed—and near-instant response for popular functions, all while mitigating frequent cold starts for low-usage functions.

B. Reducing Cold Start Delay

The cold start delay represents the initialization time required when a function is triggered from an idle state. By optimizing startup times, the Transformer model enhances responsiveness and reduces function latency, ensuring a better user experience.

To achieve this, we rely on historical invocation data to predict future demand. In our broader dataset of HTTP-triggered functions, we randomly selected 12 different invocation patterns—each referred to as a dataset—to evaluate how well our model can predict and mitigate cold starts. Two main data scenarios were tested:

- **Minute-by-minute Invocations (Figure 5 : Original Data)** : this scenario uses the original, unaltered logs where functions are invoked continuously, potentially every minute. Such fine-grained data more closely resembles real-world serverless workloads with unpredictable bursts and lulls.
- **Hourly-aggregated Invocations (Figure 6 : Altered Data)** : here, invocations are grouped by hour, simulating a coarser granularity similar to certain datasets found in the literature. While less granular, this scenario allows us to compare our model’s performance against references in other research works and to see how it handles simpler patterns.

We compared the Transformer to a standard LSTM model across multiple standard metrics—such as sMAPE (Symmetric

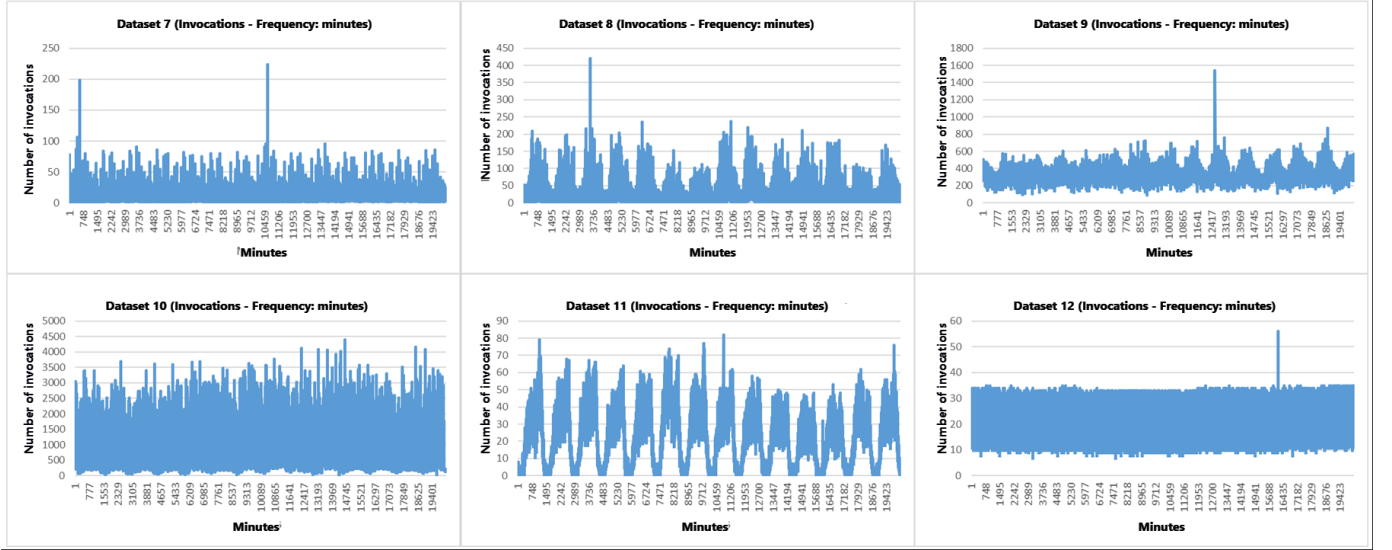


Fig. 5. Original Data : Minute-by-minute Invocations

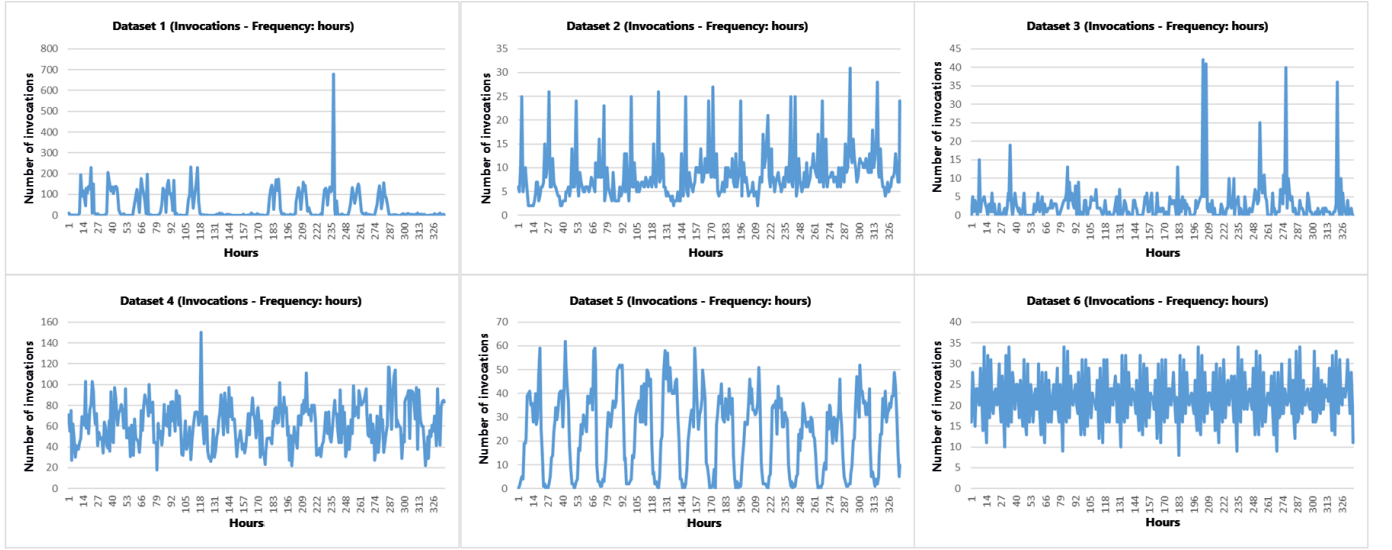


Fig. 6. Altered Data : Hourly-aggregated Invocations

Mean Absolute Percentage Error), Explained Variance, RMSE (Root Mean Squared Error), Normalized RMSE, R^2 Score, and Spearman Correlation, to capture different facets of prediction error and variance :

- Tables I & Table II illustrate performance results for cold start delay predictions, examining both hourly and minute invocation frequencies.
- Minute-Level Predictions: The Transformer achieved a lower sMAPE compared to LSTM (e.g. : Dataset 12 : Transformer sMAPE = 0.043 vs. LSTM = 0.108).
- Hourly Predictions: for altered datasets, the Transformer demonstrated superior stability, with explained variance up to 0.958 (e.g. : Dataset 12), indicating robust capture of workload variability.
- Normalized RMSE: The Transformer reduced errors in high-variability scenarios (e.g. : Dataset 10: Transformer

N. RMSE = 0.101 vs. LSTM = 0.119).

In sum, cold start delay is markedly reduced thanks to the Transformer's predictive precision, enabling a more responsive and cost-effective environment for HTTP-triggered functions.

C. Reducing Cold Start Frequency

The frequency of cold starts indicates how often a function restarts from a cold state, directly impacting resource efficiency. We tested two main configurations:

- Default (Static) configuration : by default, OpenWhisk keeps a container idle for a fixed 10-minute window. This static approach does not adapt to varying workloads. If demand patterns shift (bursts of invocations followed by long idle periods), the platform either keeps containers around too long (wasting resources) or shuts them down too soon (leading to more cold starts).

TABLE I
PERFORMANCE RESULTS - COLD START DELAY: DATASET (FREQUENCY:
HOUR)

Functions	Model	sMAPE	E.V. ^a	RMSE	N. RMSE ^b	R2 Score	S. C. ^c
Dataset 1	LSTM	1.204	-3.195	8.468	0.705	-5.607	-0.034
	Transformers	1.611	-0.781	3.659	0.365	-0.876	0.065
Dataset 2	LSTM	0.321	0.202	4.641	0.193	0.196	0.499
	Transformers	0.358	0.710	3.874	0.161	0.537	0.498
Dataset 3	LSTM	1.256	-0.546	8.982	0.249	-0.546	-0.045
	Transformers	1.446	-0.059	7.493	0.208	-0.069	0.320
Dataset 4	LSTM	0.274	0.053	18.988	0.256	0.040	0.321
	Transformers	0.229	0.250	16.931	0.228	0.247	0.602
Dataset 5	LSTM	0.368	0.797	6.767	0.140	0.796	0.762
	Transformers	0.444	0.837	6.412	0.133	0.814	0.797
Dataset 6	LSTM	0.107	0.788	2.957	0.147	0.686	0.865
	Transformers	0.103	0.893	2.352	0.106	0.835	0.957

^a E.V.: Explained Variance. ^b N. RMSE: Normalized RMSE. ^c S. C.: Spearman Correlation.

TABLE II
PERFORMANCE RESULTS - COLD START DELAY: DATASET (FREQUENCY:
MINUTE)

Functions	Model	sMAPE	E.V. ^a	RMSE	N. RMSE ^b	R2 Score	S. C. ^c
Dataset 7	LSTM	0.333	0	4.163	0.143	-0.009	0.001
	Transformers	0.201	0.717	2.368	0.081	0.700	0.659
Dataset 8	LSTM	1.187	-0.225	15.601	0.288	-0.330	-0.062
	Transformers	1.371	0.274	12.360	0.228	0.141	0.441
Dataset 9	LSTM	0.096	-0.183	53.663	0.177	-0.277	0.189
	Transformers	0.096	0.193	51.213	0.169	-0.164	0.461
Dataset 10	LSTM	0.166	0.747	196.030	0.119	0.738	0.753
	Transformers	0.128	0.805	167.645	0.101	0.802	0.777
Dataset 11	LSTM	0.425	0.458	3.518	0.152	0.450	0.704
	Transformers	0.412	0.522	3.236	0.140	0.522	0.731
Dataset 12	LSTM	0.108	0.813	3.060	0.122	0.711	0.898
	Transformers	0.043	0.958	1.179	0.047	0.958	0.974

^a E.V.: Explained Variance. ^b N. RMSE: Normalized RMSE. ^c S. C.: Spearman Correlation.

- Adaptive configuration using Transformers : our model dynamically adjusts the container inactivity window based on predicted demand intervals. The idle time is tailored to each function's actual usage pattern, resulting in a more flexible, real-time approach.

To evaluate frequency-related improvements, we selected 6 different invocation patterns relevant to this problem, each representing distinct usage profiles (sporadic vs. semi-regular vs. bursty traffic). Two key metrics were observed :

- Idle-Container Window (Figure 7) : under the default configuration, is always 10 minutes, regardless of actual demand. Under the Transformers approach, the inactivity window adapts in real time. For instance, in one dataset (call it "dataset 13"), the window varied from 14 to 22 minutes, indicating relatively stable usage with predictable idle times. Another dataset (e.g., "dataset 14") showed a narrower range (7 to 15 minutes), reflecting more frequent usage spikes. In highly irregular datasets, such as "dataset 16," the window ranged from 5 to 141 minutes, demonstrating the platform's need for maximum flexibility to avoid unnecessary cold starts.
- Number of Cold Starts (Table III) : with a fixed 10-minute window, the number of cold starts was consistently high across all tested datasets. For example, one dataset

TABLE III
COMPARISON OF INACTIVE CONTAINER WINDOW VARIATIONS AND
NUMBER OF COLD STARTS ON OPENWHISK WITH AND WITHOUT
ASSOCIATED TRANSFORMERS MODEL

Function	Platform	ICW ^a Variation	CS ^b /100
Dataset 13	OpenWhisk	fixed = 10 min	100
	OW^c + Transf^d	14-22 min	45
Dataset 14	OpenWhisk	fixed = 10 min	66
	OW^c + Transf^d	7-15 min	14
Dataset 15	OpenWhisk	fixed = 10 min	72
	OW^c + Transf^d	3-35 min	45
Dataset 16	OpenWhisk	fixed = 10 min	71
	OW^c + Transf^d	5-141 min	25
Dataset 17	OpenWhisk	fixed = 10 min	62
	OW^c + Transf^d	4-26 min	38
Dataset 18	OpenWhisk	fixed = 10 min	66
	OW^c + Transf^d	7-16 min	17

^a ICW: Idle Container Window.

^b CS: Cold start.

^c OW: OpenWhisk.

^d Transf: Transformers.

recorded 100 cold starts for 100 invocations, implying every request triggered a new container. Others ranged from 66 to 72 cold starts, all of which are suboptimal. By contrast, with dynamic adjustment via the Transformer model, cold starts dropped significantly ; In one dataset, cold starts fell from 100 to 45 (a 55% reduction). Another dataset saw an even more dramatic reduction from 66 to 14 (79% reduction). Similar improvements were observed across the board, with reductions of 65% or more in some cases.

This adaptive approach to container management clearly outperforms the static configuration. Specifically : efficient resource utilization, reduced cold start incidence, scalability and flexibility. By using Transformers to predict each function's unique demand profile, we can dynamically optimize container idle times, substantially lower cold start frequency, and improve user experience.

By combining DBSCAN clustering to segment functions by invocation pattern, Transformer-based predictions for fine-grained scheduling, and dynamic container management in OpenWhisk, this approach drastically reduces both cold start delay and cold start frequency, ultimately improving the reliability and scalability of serverless applications.

D. General Discussion of Results and Implications for FaaS Environments

The Transformer model demonstrates significant effectiveness in managing cold starts, outperforming LSTM in two critical aspects: delay and frequency. Key advantages can be summarized as follows :

- Reduced Overall Latency : by minimizing cold start delay, the Transformer enables near-instant execution for frequently invoked functions, essential for low-latency services ;
- Lower Infrastructure Costs : reduced cold start frequency ensures optimal use of cloud resources, avoiding excessive costs linked to repeated startups ;

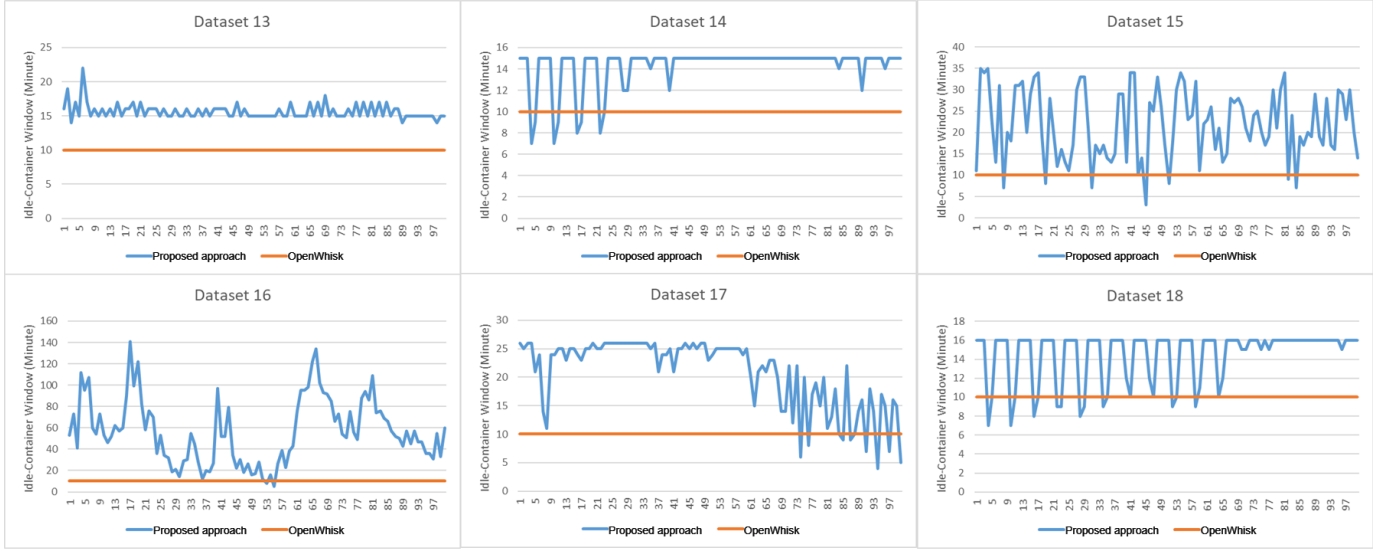


Fig. 7. Idle-Container Window

- **Enhanced Responsiveness in Dynamic Environments :** thanks to its attention mechanism and efficient temporal relationship processing, the Transformer delivers accurate predictions even in unstable workload environments, crucial for dynamic cloud applications.

These results highlight the practical implications of using Transformers in FaaS architectures. By optimizing both prediction accuracy and resource efficiency, our model provides a robust solution for highly responsive and scalable cloud services.

Transformers represent a significant advancement in cold start management for serverless environments. Their prediction and resource optimization capabilities improve both function responsiveness and resource consumption efficiency. Based on these findings, FaaS environments can leverage this architecture to enhance user experience while reducing costs, making it an ideal solution for modern cloud computing.

VI. MODEL LIMITATIONS AND EDGE CASES

Although our Transformer-based approach has generally demonstrated superior performance in reducing cold start delays and frequency, certain configurations and datasets reveal less satisfactory results, sometimes even underperforming compared to the LSTM model. This section details these specific cases and associated metrics to identify the model's limitations.

A. Cold Start delay reduction – invocations per minute

When analyzing predictions at one-minute intervals, we observed three datasets (Datasets 19, 20, and 21) where the Transformer model occasionally underperformed compared to the LSTM model. A summary of the results is in Table IV.

B. Cold Start delay reduction – invocations per hour

For invocations aggregated per hour (Datasets 22, 23, and 24), the comparison also reveals cases where the Transformer

TABLE IV
PERFORMANCE RESULTS - COLD START DELAY: MINUTE-LEVEL

Func.	Model	sMAPE	E.V. ^a	RMSE	NRM ^b	R2 Sc.	S. C. ^c
Data19	LSTM	0.837	0.045	1.436	0.239	-0.105	0.448
	Transf.	1.437	-0.385	1.784	0.297	-0.686	0.134
Data20	LSTM	0.142	0.102	13.651	0.206	0.039	0.385
	Transf.	0.139	0.024	13.876	0.210	0.007	0.176
Data21	LSTM	0.641	0.144	2.384	0.149	0.136	0.222
	Transf.	0.648	0.094	2.495	0.155	0.081	0.236

^a E.V.: Explained Variance. ^b N. RMSE: Normalized RMSE. ^c S. C.: Spearman Correlation.

model does not achieve the expected performance (see Table V).

TABLE V
PERFORMANCE RESULTS - COLD START DELAY: HOURLY

Functions	Model	sMAPE	E.V. ^a	RMSE	N. RMSE ^b	R2 Score	S. C. ^c
Dataset 22	LSTM	0.029	0.940	0.525	0.087	0.929	0.944
	Transformers	0.031	0.923	0.582	0.097	0.913	0.865
Dataset 23	LSTM	0.507	0.091	1.962	0.245	-0.102	0.383
	Transformers	0.686	-0.203	2.257	0.282	-0.203	0.395
Dataset 24	LSTM	0.829	0.418	1.375	0.196	0.418	0.679
	Transformers	0.829	0.204	1.573	0.224	0.204	0.491

^a E.V.: Explained Variance. ^b N. RMSE: Normalized RMSE. ^c S. C.: Spearman Correlation.

C. Cold Start frequency reduction

Results related to cold start frequency management also highlight some shortcomings of the Transformer-based adaptive model. The comparisons between the default OpenWhisk configuration and the OpenWhisk + Transformers configuration yield the following results (see table VI).

These observations show that in some cases, the dynamic container inactivity window adjustment algorithm failed to

TABLE VI
COMPARISON OF INACTIVE CONTAINER WINDOW VARIATIONS AND
NUMBER OF COLD STARTS ON OPENWHISK WITH AND WITHOUT
TRANSFORMERS MODEL

Function	Platform	ICW ^a Variation	CS ^b /100
Dataset 25	OpenWhisk OW^c + Transf^d	fixed = 10 min 1-25 min	33 40
Dataset 26	OpenWhisk OW^c + Transf^d	fixed = 10 min 2-23 min	71 76
Dataset 27	OpenWhisk OW^c + Transf^d	fixed = 10 min 6-12 min	42 47

^a ICW: Idle Container Window.

^b CS: Cold Start.

^c OW: OpenWhisk.

^d Transf: Transformers.

optimize resource management, leading to an increase in cold starts.

In summary, while Transformers provide notable improvements in many scenarios, these negative results highlight situations where the model has not met expectations. This calls for targeted improvements to ensure consistently strong performance across all use cases.

VII. CONCLUSION

In this research, we proposed an innovative approach to mitigate cold start issues in Function-as-a-Service (FaaS) environments, relying on advanced deep learning models, specifically Transformers. Our contributions are summarized as follows: we developed a method that anticipates and reduces cold start delays, significantly improving the responsiveness of cloud functions; our Transformer-based model outperformed traditional approaches, particularly in environments with high workload variability, by enabling more accurate predictions of future invocations and optimized container management; in terms of cold start delay reduction, our models demonstrated enhanced performance, although certain limitations, such as the granularity of invocation data, restricted the prediction accuracy in cases with short intervals.

The results of this study have significant implications for both research and practical applications in the field of cloud computing. By adopting Transformer models for the prediction and proactive management of cold starts, our work paves the way for more robust and responsive solutions in serverless environments. These improvements are particularly beneficial for applications where low latency is critical, such as real-time systems and highly interactive services. However, the observed limitations—particularly the temporal aggregation of data and the absence of certain data due to privacy concerns—highlight the importance of adapting these models to the specificities of platforms and workloads. This work also provides a solid foundation for future optimizations, particularly in reducing operational costs related to cloud resources.

The encouraging results obtained in this study suggest several areas for further exploration :

- Exploration of new Transformer architectures : testing and adapting more recent and specialized Transformer architectures could provide additional improvements for

cold start mitigation, particularly in terms of speed and accuracy ;

- Integration of reinforcement learning : adding reinforcement learning techniques would allow for dynamic, real-time cloud resource management, ensuring more responsive and optimal resource allocation ;
- Advanced resource management optimization : more sophisticated memory management mechanisms could be incorporated to further balance performance and operational costs. For example, the implementation of adaptive algorithms for maintaining containers in memory ;
- Inclusion of cross-platform data : by incorporating usage data from various FaaS platforms, as well as application metrics, it would be possible to improve the accuracy of models and tailor them more specifically to the nuances of different usage contexts.

In conclusion, our research highlights the potential of deep learning models to address the specific challenges of FaaS architectures while laying the groundwork for future innovations in proactive serverless resource management [23], [29].

REFERENCES

- [1] N. T. Noble, Y. P. Dev, and C. T. Joseph, "Machine Learning Based Techniques for Workload Prediction in Serverless Environments," in *2023 International Conference on Electrical, Electronics, Communication and Computers (ELEXCOM)*, (Roorkee, India), pp. 1–6, IEEE, Aug. 2023.
- [2] A. U. Gias and G. Casale, "COCO: Cold Start Aware Capacity Planning for Function-as-a-Service Platforms," July 2020. arXiv:2007.01222 [cs].
- [3] M. Shahradd, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider," pp. 205–218, 2020.
- [4] P. Vahidinia, B. Farahani, and F. S. Aliee, "Mitigating Cold Start Problem in Serverless Computing: A Reinforcement Learning Approach," *IEEE Internet of Things Journal*, vol. 10, pp. 3917–3927, Mar. 2023. Conference Name: IEEE Internet of Things Journal.
- [5] S. Eismann, J. Scheuner, E. v. Eyk, M. Schwinger, J. Grohmann, N. Herbst, C. L. Abad, and A. Iosup, "The State of Serverless Applications: Collection, Characterization, and Community Consensus," *IEEE Transactions on Software Engineering*, vol. 48, pp. 4152–4166, Oct. 2022. Conference Name: IEEE Transactions on Software Engineering.
- [6] T. N. Nguyen, "Managing Cold-start in The Serverless Cloud with Temporal Convolutional Networks," Apr. 2023. arXiv:2304.00396 [cs, eess].
- [7] "The CNCF takes steps toward serverless computing," Feb. 2018.
- [8] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions," Oct. 2023. arXiv:2310.08437 [cs].
- [9] A. P. Jegannathan, R. Saha, and S. K. Addya, "A Time Series Forecasting Approach to Minimize Cold Start Time in Cloud-Serverless Platform," in *2022 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, pp. 325–330, June 2022.
- [10] A. Mampage, S. Karunasekera, and R. Buyya, *A Deep Reinforcement Learning based Algorithm for Time and Cost Optimized Scaling of Serverless Applications*. Aug. 2023.
- [11] A. Kumari and B. Sahoo, "ACPM: adaptive container provisioning model to mitigate serverless cold-start," *Cluster Computing*, vol. 27, pp. 1–28, May 2023.
- [12] K. Tirkey, A. Kumari, S. Mohanty, and P. B. Sahoo, "A Novel Function Fusion Approach for Serverless Cold Start," in *2023 International Conference on Communication, Circuits, and Systems (IC3S)*, pp. 1–5, May 2023.
- [13] D. Barcelona-Pons and P. García-López, "Benchmarking Parallelism in FaaS Platforms," *Future Generation Computer Systems*, vol. 124, pp. 268–284, Nov. 2021. arXiv:2010.15032 [cs].
- [14] I. Matta, "Function-as-a-Service: From an Application Developer's Perspective," Jan. 2021.

- [15] M. Steinbach, A. Jindal, M. Chadha, M. Gerndt, and S. Benedict, "TppFaaS: Modeling Serverless Functions Invocations via Temporal Point Processes," *IEEE Access*, vol. PP, pp. 1–1, Jan. 2022.
- [16] J. Manner, M. Endreß, T. Heckel, and G. Wirtz, "Cold Start Influencing Factors in Function as a Service," in *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*, pp. 181–188, Dec. 2018.
- [17] S. Lee, D. Yoon, S. Yeo, and S. Oh, "Mitigating Cold Start Problem in Serverless Computing with Function Fusion," *Sensors*, vol. 21, p. 8416, Jan. 2021. Number: 24 Publisher: Multidisciplinary Digital Publishing Institute.
- [18] "apache/openwhisk," June 2024. original-date: 2016-02-18T21:16:19Z.
- [19] "AWS | Lambda - Service PaaS de calculs distribués."
- [20] "lambda-warmer," Nov. 2023.
- [21] X. Liu, J. Wen, Z. Chen, D. Li, J. Chen, Y. Liu, H. Wang, and X. Jin, "FaaSLight: General Application-level Cold-start Latency Optimization for Function-as-a-Service in Serverless Computing," *Transactions ACM sur le génie logiciel et la méthodologie*, vol. 32, pp. 119:1–119:29, July 2023.
- [22] "Cloud Functions."
- [23] S. Agarwal, M. A. Rodriguez, and R. Buyya, "A Reinforcement Learning Approach to Reduce Serverless Function Cold Start Frequency," in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pp. 797–803, May 2021.
- [24] S. Agarwal, M. A. Rodriguez, and R. Buyya, "A Deep Recurrent-Reinforcement Learning Method for Intelligent AutoScaling of Serverless Functions," Aug. 2023. arXiv:2308.05937 [cs, eess].
- [25] A. Kumari, B. Sahoo, and R. K. Behera, "Mitigating Cold-Start Delay using Warm-Start Containers in Serverless Platform," in *2022 IEEE 19th India Council International Conference (INDICON)*, pp. 1–6, Nov. 2022. ISSN: 2325-9418.
- [26] K. Rasul, "kashif/pytorch-ts," June 2024. original-date: 2022-03-21T15:40:42Z.
- [27] "blog/time-series.md at main · huggingface/blog."
- [28] "AzurePublicDataset/AzureFunctionsDataset2019.md at master · Azure/AzurePublicDataset."
- [29] S. Agarwal, M. A. Rodriguez, and R. Buyya, "Reinforcement Learning (RL) Augmented Cold Start Frequency Reduction in Serverless Computing," Aug. 2023.